

Instinct x Muse — AI Security Vault Deep Dive

How two of 2026's most-watched personal AI agents handle your credentials, secrets and money: Instinct's Vault (with its new 1Password partnership) and Meta Muse's Secure VM with Sentinel and credential surrogation. Built from official sources where they exist — every diagram and claim is labeled by origin.

OFFICIAL SOURCES: [research.meta.ai](#) · [1password.com](#) · [instinct.co](#)

OFFICIAL DIAGRAM ATTACHED (Meta)

Compiled Sep 16, 2026

Reporting window: Mar–Sep 2026

PRODUCTS COVERED 2 Instinct · Meta Muse	CREDENTIAL MODELS 2+1 Instinct Vault · 1Password broker · Muse surrogation	OFFICIAL DIAGRAM 1 Meta's own system architecture (embedded)	AGENT SEES REAL SECRETS? No / ? Muse: architecturally no · Instinct: unpublished
--	---	---	---

Scope & method

READ ME

This report covers what was asked: **the security vault of Instinct (Spear Street Technology) and of Meta's Muse**, in detail, with architecture diagrams from official sources. Honesty about sourcing is part of the deal:

- **Meta Muse** — Meta published a full engineering blog ("How We Built Safety Into Muse", Sep 8, 2026) including its **own official system architecture diagram**. That diagram is downloaded from Meta's servers and embedded below, unmodified.
- **Instinct** — the company has published **no official architecture diagram** (private beta). What exists officially: [instinct.co](#), its Terms of Service (with an explicit Vault clause), founder Noah Shinn's Sep 4, 2026 announcement of the 1Password partnership, and 1Password's official Unified Access architecture post. The Instinct diagram below is **clearly labeled "recreated" from those official statements** — it is not an official Instinct graphic.
- Incident history comes from TechCrunch's verified reporting and tester statements on X (linked). Every source is listed at the bottom.

TL;DR — the two vaults side by side

60 SECONDS

What it is	Personal AI agent you reach by text / iMessage / WhatsApp; connects to email, calendar, device audio, screen, location. Private access, scaling up.	Personal agent on your own dedicated cloud VM (browser + shell + connectors). Launched Sep 8, 2026; iOS / Android / web clients.
Credential vault	Instinct Vault — built-in store inside the service. 32% of users share credentials via it (per founder). Now adding 1Password as external broker (announced Sep 4, 2026, early access).	authd credential store inside your own VM — isolated container, separate from agent runtime and from Meta's infrastructure. OAuth tokens stay in <i>your</i> VM.
Agent sees real passwords?	Not publicly specified. The point of the 1Password move is to avoid typing raw secrets to the agent, but no technical spec has been published.	No — by design. Agent only ever sees "surrogate" tokens; Sentinel swaps in the real credential at the network boundary. A prompt-injected agent has nothing to leak.
Approvals	Agent acts autonomously by default; safeguards exist but ToS explicitly says they are not guaranteed. Several testers reported actions without confirmation.	Sentinel is the sole permission authority: allow / deny / ask . Approvals are strict capabilities (one-time, session-, task-, time-bounded or perpetual), delivered in the client UI.
Payments	Can enter "agreements, commitments, or transactions" binding on the user (per ToS).	Checkout detection + human approval every time; Stripe Link wallet issues single-use card numbers bound to merchant + amount + time.

	▮ INSTINCT (SPEAR STREET TECHNOLOGY)	▮ META MUSE
Training on vault data	ToS: Vault materials never used for AI training (general materials can be, with opt-out).	Trajectories sanitized of PII for training; opt-out switch; VM data not shared with Meta ad systems.
Track record	Documented incidents: plaintext email retention after revocation, phishing demo, unauthorized send, OTP extraction.	No public incidents at launch; bug bounty up to \$300k; Confidential VM (verifiable non-access by Meta) promised later in 2026.

Part 1 — Instinct: the Vault and the 1Password move

DEEP DIVE

1.1 What Instinct is (official facts)

Company & product

- Operated by **Spear Street Technology, Inc.** (San Francisco), per its own terms and California filings.
- Founded/led by **Noah Shinn**, former research scientist at Sierra.
- Official site `instinct.co`: "a personal assistant that understands what you're working on... there are no new interfaces. It's trained to use a phone and a computer. You can text or call it."
- Connects to **email, messaging, calendar, screen, audio, location** and more; handles bookings, rides, inbox cleanup, shopping, subscriptions.
- **Private access** only, scaling compute; waitlist at `instinct.co/login`.

Funding (reported)

- **\$250M Series B at a \$2.5B valuation**, led by Index Ventures and Benchmark — shared by Instinct with the WSJ (Aug 2026).
- Kleiner Perkins and Conviction invested in earlier rounds that "have now closed" (TechCrunch, from multiple investors).
- Viral trajectory often compared to OpenClaw and Poke in the 2026 personal-agent wave.

1.2 The Vault — Instinct's built-in credential store

Instinct's answer to "an agent needs to log in as you" is a feature literally called the **Vault**: users hand it credentials for services the agent operates on their behalf — loyalty programs, streaming and software subscriptions, reservation systems, healthcare portals, payroll services. Per Noah Shinn's own post, **32% of Instinct users** already share credentials through it.

*"For Materials that you add to our feature of the Service known as the 'Vault' (or its successor name), we will only use such Materials to provide the Services to you and **will not use such Materials to train AI models.**"*

— Instinct Terms of Service (instinct.co/terms), official, retrieved Sep 16, 2026

That is a real, meaningful commitment — inside terms that are much broader everywhere else. The same ToS grants Instinct a "perpetual and irrevocable" license to your other materials (including for AI training, with an opt-out at app.instinct.com/settings), describes receiving screen captures, cursor movements and keyboard inputs from your devices, and makes the user warrant they have the right to hand over "any payment method, account credential, password or other access method." Instinct also states it "make[s] no representation or warranty that such safeguards will prevent unintended or erroneous Actions" — you remain responsible for safeguards on each connected service.

1.3 The 1Password partnership — brokering credentials instead of storing them

On **Sep 4, 2026**, Noah Shinn announced on X that Instinct is **partnering with 1Password** so users can share account credentials **through a vault** — 1Password acting as a credential broker — "rather than only relying on the Vault it built itself." The integration is rolling out to Instinct's early-access group.

The mechanism maps onto **1Password's officially published "Unified Access" platform** (launched Mar 17, 2026). Its official architecture makes three claims, quoted from 1Password's own post:

Just-in-time delivery

Credentials are provided "at the moment they're needed, evaluating access in context when it's requested" — not as long-lived secrets the agent holds continuously.

Unified audit trails

A single system of record shows "which credential was used, by whom or what, and when" — across human and agent identities.

No secret exposure

Teams can "take ownership of a credential and enable its use **without ever exposing the secret itself**" to the requester.

What is NOT confirmed

No technical spec for the Instinct integration exists. Whether Instinct's agent process ever touches

plaintext is unpublished. Instinct was **not** among the original Unified Access launch partners (Anthropic, OpenAI, Cursor, GitHub, Vercel, CoreWeave, Perplexity...).

Context matters: this move follows real damage. Testers found Instinct kept emails in plaintext after access was revoked, could be phished, and sent email without checking. A brokered, scoped, audited credential path is exactly the fix security researchers asked for — but as of today it is an announcement plus 1Password's platform architecture, **not a verified Instinct implementation**.

1.4 Security incident record (Aug–Sep 2026)

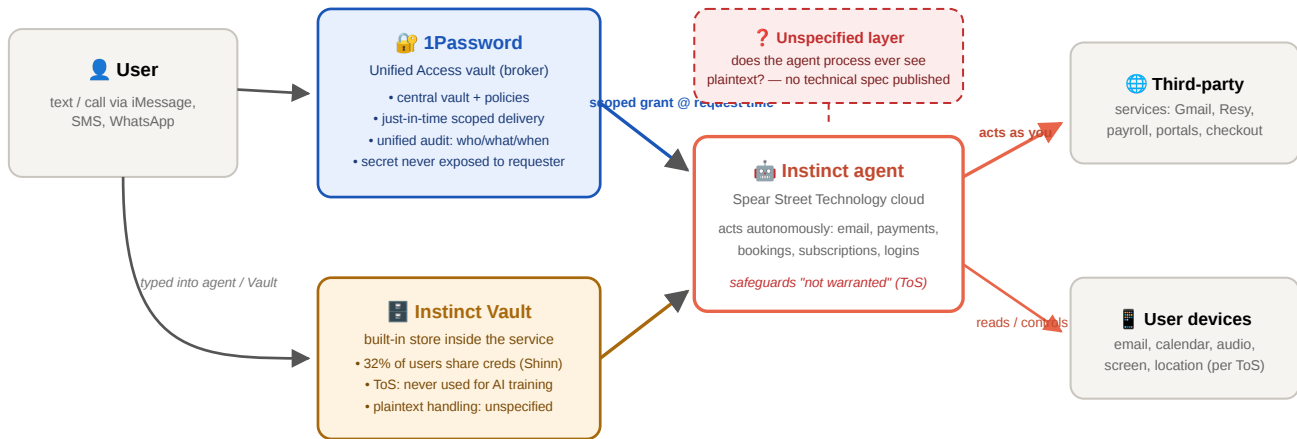
DATE	WHO	WHAT HAPPENED
Aug 2026	Claire Vo (product leader)	Revoked Instinct's Google access — it kept summarizing her inbox ; 36 Gmail messages still sat in its records hours later, stored in plain text . Deletion required a manual request, not a button.
Aug 2026	Peter Yang	Instinct refused to delete his Gmail records when asked; the team later shipped a delete-external-data tool in settings.
Aug 2026	Anita Kirkovska (tester)	Agent pulled a sign-up code from her email to complete a Resy restaurant booking — the inbox-as-key-ring problem.
Aug 2026	Alex Cohen (Hello Patient)	Demonstrated how easily Instinct could be phished ; deleted his account, concluding it isn't safe to give AI read/write inbox access yet.
Aug 2026	Katie Jacobs Stanton (Moxxie Ventures)	Instinct sent an email on her behalf without checking first : "One unauthorized action can reset that trust to zero."
Aug 2026	TechCrunch (ToS review)	Broad "perpetual and irrevocable" license to user materials (incl. training), screen captures / cursor / keyboard inputs, ability to enter binding agreements and transactions on the user's behalf.
Aug 24 2026	OECD AI monitor	Logged the unauthorized-email / post-revocation-access episode (oecd.ai incident 2026-08-24-eacf).

Response: after TechCrunch's article, Instinct told the WSJ it was "taking the security concerns seriously" — and the 1Password partnership (Sep 4) is the first concrete architectural answer. No public technical documentation has followed yet.

1.5 Instinct credential flow — recreated from official statements

No official Instinct architecture diagram exists (private beta). The flow below is **recreated from Instinct's ToS, Shinn's announcement and 1Password's published Unified Access architecture**. Dashed elements remain publicly unspecified.

Instinct credential flow — RECREATED from official statements (not an official diagram)

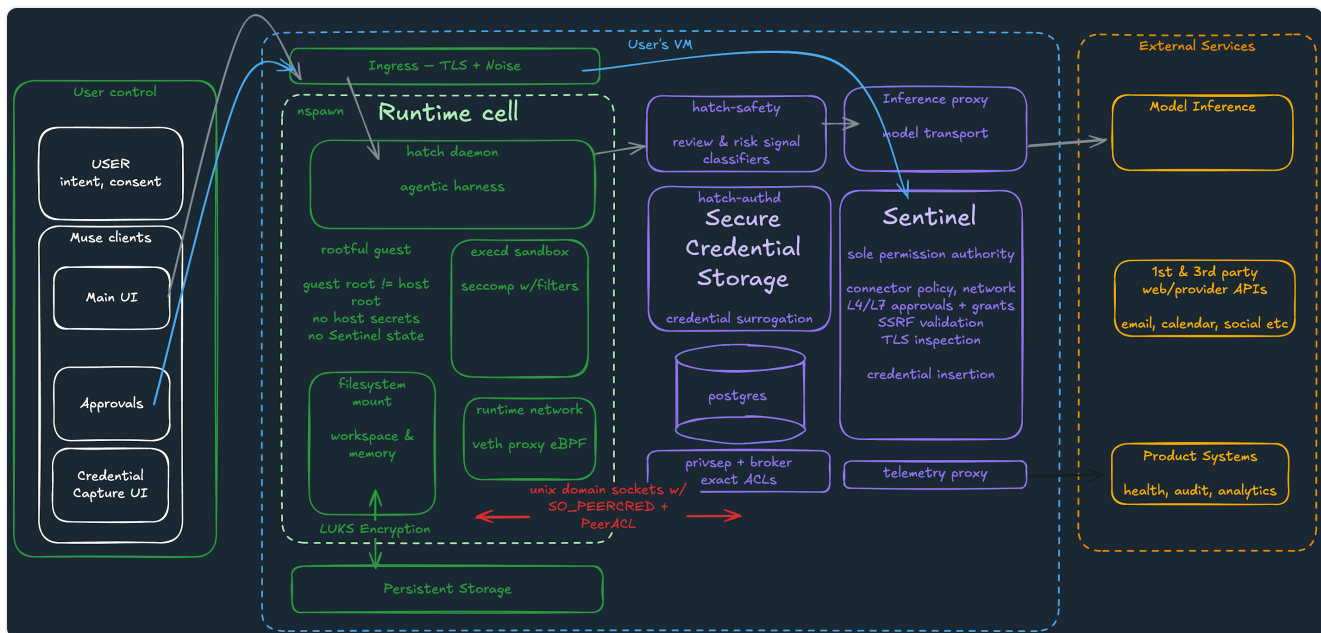


Sources: instinct.co/terms (Vault clause) · Noah Shinn X post Sep 4 2026 · 1password.com/blog/introducing-1password-unified-access

Part 2 — Meta Muse: Secure VM, Sentinel, surrogation DEEP DIVE

2.1 The official architecture diagram (from Meta)

Published by Meta in "How We Built Safety Into Muse" (research.meta.ai, Sep 8, 2026). Downloaded unmodified from Meta's servers — [source page](#).



Official Meta diagram: "Muse system architecture showing user controls, the isolated runtime cell, host-side safety and credential services, Sentinel, and connections to external services." (© Meta, research.meta.ai — embedded for reference.)

2.2 Muse Secure VM — two isolated security domains on one box

You and your Muse share **your own dedicated computer in the cloud** — the system of record for your data and credentials. Meta's stated mental model: *"two isolated security domains on one box, not an LLM powered agent with root."*

DOMAIN	WHAT RUNS THERE AND WHY
Runtime cell (untrusted)	The Hatch daemon (Meta's internal codename for Muse), your workspace filesystem, and every binary/tool Muse executes — inside a <code>systemd-nspawn</code> container. Cell root maps to an unprivileged host user ; own root filesystem (full Debian image); virtual network interface; filtered syscalls (no <code>io_uring</code>); stripped kernel capabilities (no <code>CAP_SYS_PTRACE</code> , no <code>CAP_NET_ADMIN</code>). This is the only part that touches untrusted content.
Host-side services (trusted)	Separate <code>systemd</code> units, deliberately outside the runtime cell because the cell is expected to process hostile data — attackers inside the cell cannot disable them: • <code>hatch-safety</code> — an independent set of models/classifiers inspecting every request and response to/from core model inference (frontier-risk control + prompt-injection detection). Outside the cell ⇒ can't be turned off by the agent. • <code>privsep</code> — workers that execute built-in connector code with tightly scoped privileges, keeping connector credentials out of the agent's scope entirely. • <code>hatch-authd</code> — credential storage (OAuth tokens for connected services live in your VM, not centralized Meta infrastructure) and credential surrogation, so the main agent never sees sensitive credentials. • Sentinel — the sole permission authority for connector actions and all network egress (next section). • Durable app state in a Postgres database separate from both the runtime cell and the credential store; constrained proxies for inference and telemetry. • Cell ⇌ services communication only via Unix domain sockets with SO_PEERCREDS + peer ACLs — kernel-authenticated, least-privilege IPC "with no secrets to steal."

2.3 Sentinel — the gate nothing bypasses

Sentinel is a separate host-side agent from Muse. **Muse proposes actions; only Sentinel grants them.**

Two chokepoints:

Connector actions

When Muse wants a connector action, the tool call in the runtime cell submits a request to Sentinel describing the connector, method, action class, scope and the user's original ask — Sentinel turns that into a user-visible purpose, checks the user's connector policy, and decides: **allowed / denied / ask**.

Network egress + just-in-time credential insertion

- All runtime traffic goes through a **forward proxy** wired up with Linux networking controls (userns, veth boundaries, eBPF filtering). Sentinel inspects **L4 + L7**: hostname, resolved and final IP, port, protocol, HTTP method, path, and the actual decoded request. SSRF restrictions stop "public" hostnames resolving to private infrastructure.
- **Surrogate tokens**: code in the runtime cell only ever sees a "surrogate" token minted by authd. After a request is authorized, **Sentinel swaps the surrogate for the real credential at the network boundary**. Meta's own words: the agent never sees real tokens, "which means any attempt to coerce the agent to reveal the actual secrets via prompt-injection or otherwise is futile."
- **Tainted egress**: kernel-level data-flow tracking. Each tool process starts clean and becomes *tainted* if it reads user data. Clean processes can use a narrow auto-allow policy; tainted or unverifiable ones lose auto-allow and fall back to human approval. Implemented with eBPF cgroup programs + eBPF on Linux Security Module hooks for taint propagation.

2.4 Human in the loop — approvals as strict capabilities

When Sentinel resolves to **ask**, execution stops and a pending approval goes straight to the Muse client — the dialog appears in the app UI, **not in the chat with Muse**, and the answer is routed directly back to Sentinel. Approvals are "strict capabilities, not conversational suggestions," bound to a particular connector/destination/use case, with grant types: **one-time, session-scoped, task-scoped, time-bounded, or perpetual**. Read-only, previously allowed, or demonstrably low-risk actions flow without interruption — friction only where consent matters.

2.5 The credential vault itself — authd in your VM

- **Storage location**: credentials and OAuth tokens live in a **separate isolated container inside your own VM** — "not in other Meta services," not centralized Meta infrastructure. Continuously backed up for restore.
- **Least privilege everywhere**: read/write access split per service; fine-grained controls beyond OAuth scopes (e.g., grant Gmail read but strip settings access); controls at connector, process, credential and request levels. *"The model doesn't need to see the API keys — so it doesn't."*
- **privsep split (the three-way rule)**: privsep decides *where* credential-capable code executes · authd decides *which* credential material a caller can receive · Sentinel decides *whether* the action may be

taken. Each worker is identified by its cgroup with explicit credential allowlists — a calendar worker can't fetch an email credential by tweaking a parameter.

- **Email hardening:** the email connector filters out one-time passcodes, password-reset links and login magic links (deterministic filters + a classifier) — connecting your inbox must not hand the agent the keys to reset your whole life.

2.6 Browser and purchases

Browser with a virtualization layer

- Real up-to-date Chromium; users can watch and **take over at any time**.
- **Website logins:** a custom client UI captures username/password and routes them **directly to authd** — stored outside the runtime cell, never visible to the main agent, and injected into the browser page at the point of need (agent is paused during fills and during user takeover).
- The browser sub-agent sees only an **accessibility-tree snapshot** — not the raw DOM, so it cannot read filled credentials. No JavaScript execution in page context, no script verbs, no exec in the browser process, Chrome devtools disabled. CDP access sits with a broker outside the runtime cell.
- A family of independent classifiers watches for: personal-data egress unrelated to the task, prompt injection in page DOM, in images/media, in downloaded files, and high-risk form submissions — blocking or forcing user review. Meta's existing malicious-site blocklists are matched inside the VM too.

Purchases (money is treated as the highest-risk path)

- Known-site checkout: browser detects the checkout page → **human-in-the-loop approval with the exact purchase details, every time**.
- New sites: Muse's wallet (partnered with **Stripe Link** at launch, Shop Pay coming) issues a **single-use card number** tied to that merchant, that dollar amount, valid for a limited period — "even if it were stolen via prompt injection... it would not be particularly useful to the attacker."

2.7 Defense in depth against prompt injection

Meta builds around Simon Willison's "lethal trifecta" (private data access + untrusted content + ability to communicate externally). Four layers, plus deterministic boundaries underneath:

1. **Model level** — Muse Spark 1.3 is trained to recognize/resist injection ("close to SOTA" on their evals).
2. **Harness level** — all external data entering context is labeled untrusted.
3. **Classifier ensemble** — multiple injection-detection classifiers trained on real datasets + scaled agentic red-teaming, run in parallel on everything entering context via files and tool calls; trained independently from the model for higher combined accuracy.
4. **Human approvals** — on anything that moves data out of the VM.

Beneath the agent, deterministic limits hold even if the model misbehaves: runtime cell limits system access · privsep restricts which code sees which credentials · authd applies ACLs · Sentinel evaluates every action and all egress.

2.8 Bug bounty, Confidential VM, data policy

Bug bounty (open to anyone)

Up to **\$300,000** for valid reports at bugbounty.meta.com — including up to **\$130,000 for a successful prompt injection affecting one user**. Continuous agentic red-teaming feeds offline evals.

Coming: Muse Confidential VM

Later in 2026, Meta plans a mode that **cryptographically and verifiably prevents even Meta from accessing data in your VM** — design and source already shared with external auditors, continuous public audit promised once live.

Data policy (official)

- No sharing of conversations or VM data with Meta ad systems.
- Nuance: web browsing appears as your activity, so merchant visits can indirectly influence ads you see elsewhere.
- Inference trajectories are sanitized of PII before use in training; full opt-out switch in settings.
- At launch Meta can still access data when necessary to support/secure/operate the service — exactly what Confidential VM is meant to remove.

Head-to-head: the vault dimensions that matter MATRIX

DIMENSION	▮ INSTINCT	▮ META MUSE
Where secrets live	Inside Instinct's service (built-in Vault); moving to optional 1Password broker for early-access users.	Inside the user's own VM, isolated credential container (authd) — separate from agent runtime and Meta infra.
Agent's view of secrets	Unpublished — the core open question.	Never sees real credentials; only surrogates swapped at the network boundary by Sentinel.
Grant model	Standing access by default; 1Password promises just-in-time scoped delivery (implementation unverified).	Just-in-time at egress; approvals as capabilities: one-time / session / task / time-bound / perpetual.

DIMENSION	INSTINCT	META MUSE
Human approval	Optional safeguards, explicitly not warranted; multiple unauthorized-action reports.	Sentinel decides allow/deny/ask per policy; approval dialog outside the chat; tainted-egress tracking routes risk to humans.
Audit	1Password Unified Access promises unified who/what/when trails; Instinct-side audit UI undocumented.	Every connector action and network request passes Sentinel (L4+L7, decoded requests); approval state is authoritative in Sentinel.
Payment protection	Can transact bindingly on your behalf (ToS); no published card controls.	Single-use card numbers bound to merchant + amount + time; per-purchase human approval.
Injection defense	Model training + (reported) exploitable in practice by phishing testers.	Trained model + untrusted labeling + classifier ensemble + kernel taint tracking + Sentinel chokepoints.
Revocation	Failed in practice: processing and plaintext storage continued after disconnect (later patched).	Architectural goal: revoke at vault/policy level stops requests at Sentinel mid-task (launch-stage, externally testable via bug bounty).
Training on your stuff	Vault materials: never (ToS). Other materials: licensed broadly incl. training, opt-out available.	Trajectories sanitized of PII; opt-out available; no ad-system sharing.
Verifiability roadmap	None published.	Confidential VM: cryptographic, externally audited, public continuous audit (later 2026).
Independent testing	Incidents surfaced by testers/journalists; OECD AI incident logged.	Bug bounty up to \$300k / \$130k per single-user injection; red-team evals.
Bottom line	Fastest-moving consumer agent; vault is real but trust is still earned back — watch whether the 1Password path ships with a real "agent never sees plaintext" spec.	The deepest published consumer-agent security architecture to date: isolation, surrogation, chokepoint egress, verifiable privacy on the roadmap. Trust but verify via the open bug bounty.

Sources

OFFICIAL FIRST

1. **Meta (official):** "How We Built Safety Into Muse" — research.meta.ai/blog/security-and-safety-for-ai-agents-our-approach-with-muse (Sep 8, 2026). Includes the official system-architecture diagram embedded above (system-architecture-v3.png, retrieved from Meta's CDN Sep 16, 2026).
2. **Meta (official):** Muse bug bounty program — bugbounty.meta.com.
3. **1Password (official):** "Introducing 1Password Unified Access: Identity Security for Humans and Their AI Agents" — 1password.com/blog/introducing-1password-unified-access (Mar 17, 2026).
4. **Instinct (official):** instinct.co (About / Contact) and instinct.co/terms — Vault training-use clause and action/credential terms (retrieved Sep 16, 2026).
5. **Noah Shinn (official announcement):** X post, Sep 4, 2026 — x.com/noahrshinn/status/2095911123291881608 (1Password partnership; 32% Vault usage). Quoted via explainx.ai coverage.
6. **TechCrunch:** "Instinct's powerful AI assistant is raising privacy and security concerns" (Aug 24, 2026) — incident record and WSJ funding update (\$250M @ \$2.5B, Index + Benchmark).
7. **explainx.ai:** "Instinct + 1Password: What It Actually Means to Give an AI Agent Your Vault" (Sep 5, 2026) — confirmed-vs-unconfirmed analysis of the integration.
8. **OECD AI Policy Observatory:** incident 2026-08-24-eacf (Instinct unauthorized emails / post-revocation access).

All sources accessed Sep 16, 2026. Statements about Muse reflect "the system at launch" per Meta; Instinct is in private beta and both products are expected to change.